

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и компьютерной техники

Лабораторная работа №4
по дисциплине «Основы программной инженерии»

Вариант: 321500

Преподаватель:
Егошин Алексей Васильевич

Выполнил:
Храбров Артём Алексеевич
Группа: Р3215

Санкт-Петербург, 2026

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	2
ЗАДАНИЕ	3
Бин для вычисления среднего интервала между кликами	4
Бин считающий число точек	4
Мониторинг с помощью JConsole	5
Мониторинг и профилирование с помощью VisualVM	7
<i>Поиск и устранение утечки памяти в приложении</i>	9
ВЫВОД	12

ЗАДАНИЕ

1. Для своей программы из [лабораторной работы #3](#) по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если пользователь совершил 2 "промаха" подряд, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить количество классов, загруженных в JVM в процессе выполнения программы.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в [программе](#). По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Бин для вычисления среднего интервала между кликами

```
package lab4.backend.utils;

import jakarta.enterprise.context.ApplicationScoped;

@ApplicationScoped
public class ClickInterval implements ClickIntervalMBean {
    private long totalIntervalsSum = 0;
    private int intervalsCount = 0;
    private Long lastClickTime = null;

    public synchronized void recordClick() {
        long currentTime = System.currentTimeMillis();

        if (lastClickTime != null) {
            long currentInterval = currentTime - lastClickTime;
            totalIntervalsSum += currentInterval;
            intervalsCount++;
        }

        lastClickTime = currentTime;
    }

    @Override
    public long getAverageInterval() {
        if (intervalsCount == 0) return 0;
        return totalIntervalsSum / intervalsCount;
    }
}
```

Бин считающий число точек

```
package lab4.backend.utils;

import jakarta.enterprise.context.ApplicationScoped;

import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;

@ApplicationScoped
public class HitsCounter extends NotificationBroadcasterSupport implements
HitsCounterMBean {
    private int total = 0;
    private int hits = 0;
    private int missCount = 0;
    private long notificationSequence = 1;

    public void addPoint(boolean isHit) {
        total++;
        if (isHit) {
            hits++;
            missCount = 0;
        } else {
            missCount++;
            if (missCount == 2) {
                Notification notification = new Notification(
                    "TwoMissesInARow",
                    "PointsCounterMBean",
                    notificationSequence++,
                    "Пользователь промахнулся дважды подряд!"
                );
            }
        }
    }
}
```

```

        );
        sendNotification(notification);
        missCount = 0;
    }
}

@Override
public int getTotalPoints() {
    return total;
}

@Override
public int getHitPoints() {
    return hits;
}
}

```

Мониторинг с помощью JConsole

Сначала поднял бэкенд лабораторной работы. Потом запустил JConsole и подключился к процессу сервера. В дереве появился отдельный раздел с моими бинами.

У первого бина посмотрел, сколько точек попало в область и сколько всего было поставлено. У второго средний интервал между кликами в миллисекундах.

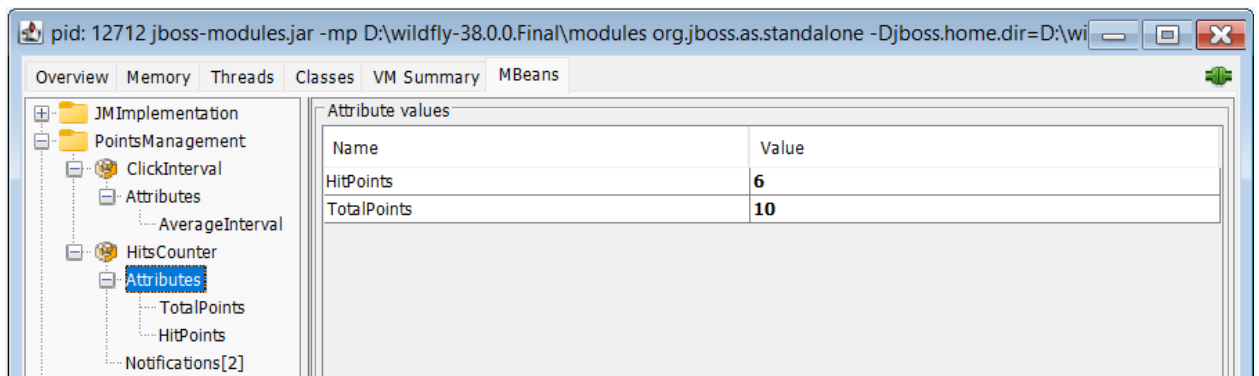


Рисунок 1 – Атрибуты MBean-компонентов HitsCounter и ClickInterval

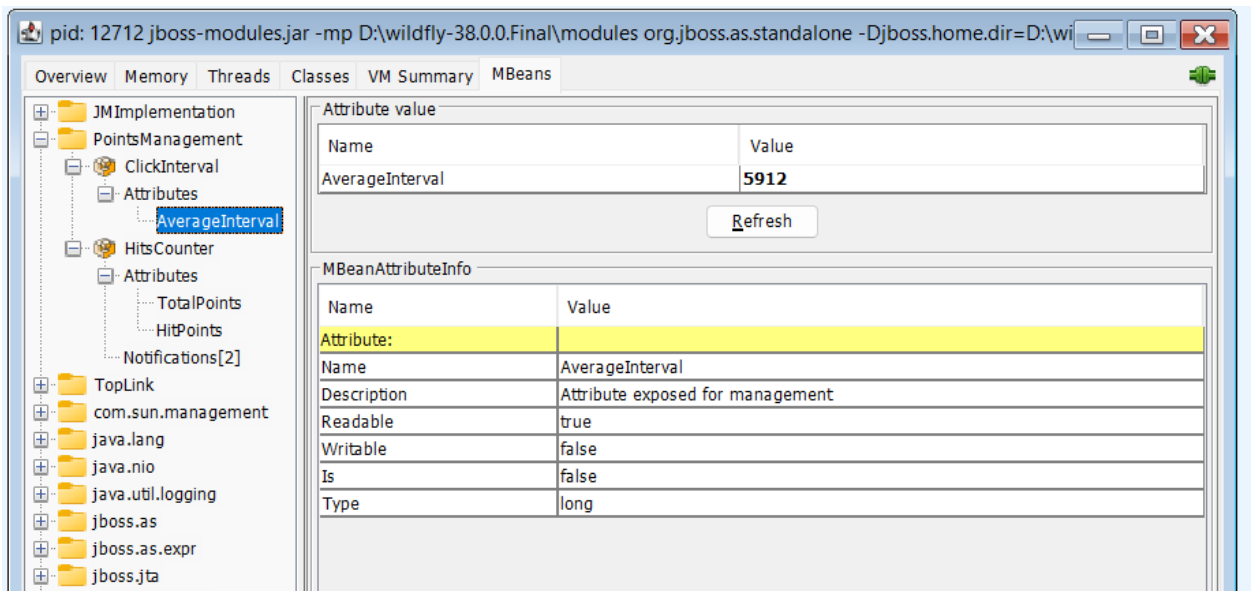


Рисунок 2 – Подробная информация об атрибуте *AverageInterval*

Дальше подписался на уведомления от первого бина и вернулся в приложение, чтобы пару раз промахнуться подряд. Сразу пришли два уведомления о том, что пользователь промахнулся два раза подряд.

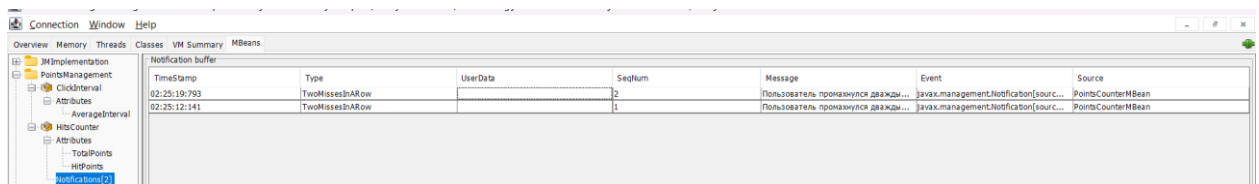


Рисунок 3 – Уведомления *TwoMissesInARow*

Посмотрел вкладку с графиком количества загруженных классов. На момент проверки было загружено чуть больше 22 тысяч классов, а выгрузилось всего около десятка. По графику видно, что почти все классы подгружаются сразу при старте сервера, а потом их число почти не меняется.

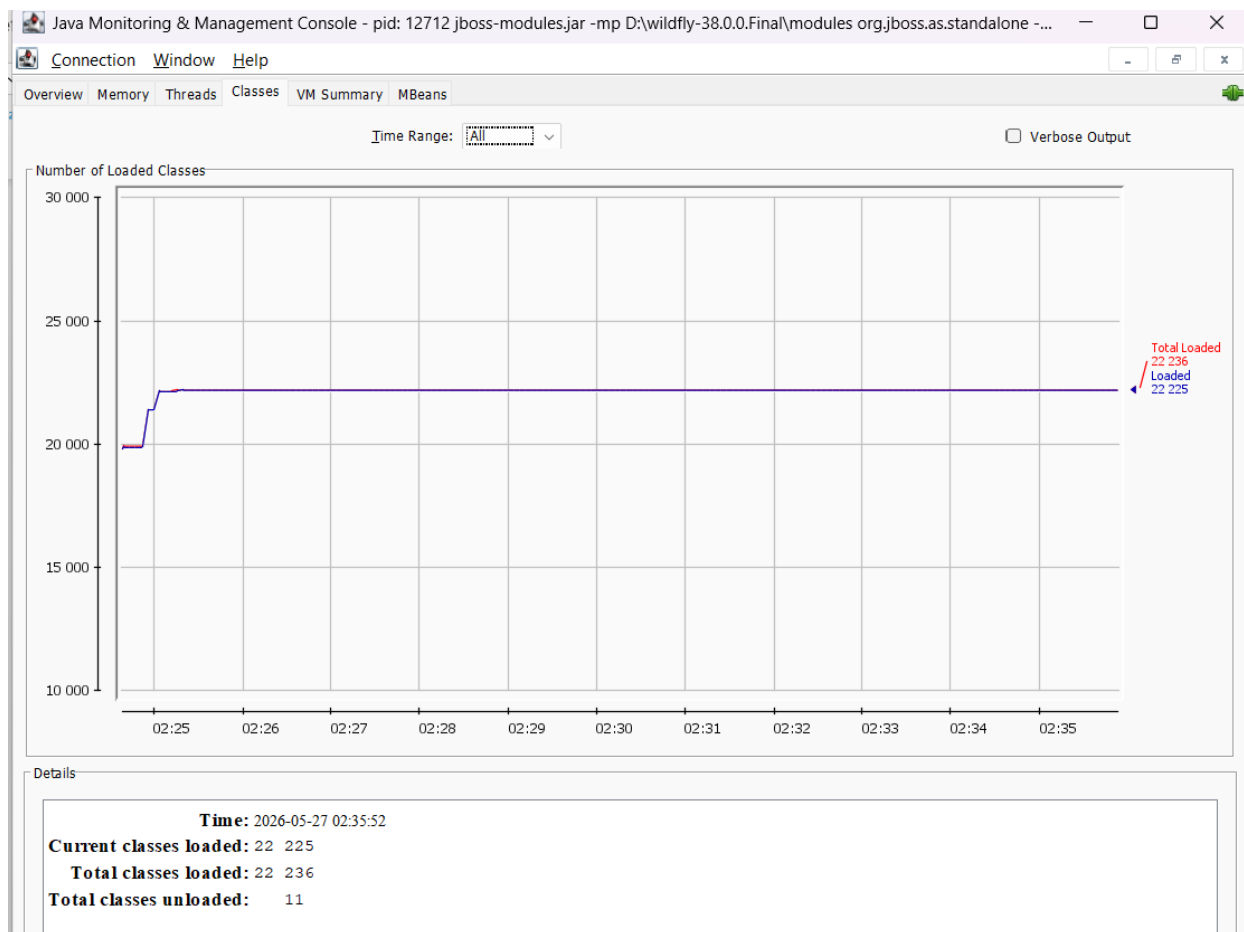


Рисунок 4 – График числа загруженных классов JVM

В итоге через эту утилиту получилось без проблем подключиться к работающему приложению, найти свои бины, посмотреть и обновить их значения по живым запросам, поймать уведомления о промахах и оценить, как ведёт себя сервер по числу загруженных классов.

Мониторинг и профилирование с помощью VisualVM

Подключился к тому же серверу, но уже другой утилитой. Она кроме значений бинов умеет ещё строить по ним графики во времени, а также показывать, куда уходит память приложения.

Нашёл те же два бина, включил для их значений построение графиков и пошёл кликать по координатной плоскости. На графиках сразу видна разница: общий счётчик точек растёт с каждым кликом, а счётчик попаданий увеличивается только тогда, когда точка попадает в область.



Рисунок 5 – Графики изменения атрибутов HitPoints и TotalPoints с течением времени

Дальше запустил профилирование памяти. В списке классов, отсортированном по объёму занимаемой памяти, на первом месте с большим отрывом оказались массивы байтов, занимающие почти четверть всей занятой памяти.

Sampler

Settings

Sample:

CPU

Memory

Stop

Status: memory sampling in progress

Heap histogram

Per thread allocations

Results:

Collected data:

Snapshot

Name	Live Bytes	Live Objects	Perform GC	Heap Dump
byte[]	30 286 872 B (24.9 %)	257 833 (9.8 %)		
java.lang.Object[]	15 485 800 B (12.8 %)	418 453 (15.9 %)		
java.util.TreeMap\$Entry	9 767 920 B (8 %)	244 198 (9.3 %)		
java.util.HashMap\$Node	7 515 872 B (6.2 %)	234 871 (8.9 %)		
java.lang.String	5 361 384 B (4.4 %)	223 391 (8.5 %)		
int[]	5 010 752 B (4.1 %)	18 023 (0.7 %)		
java.util.HashMap\$Node[]	4 403 568 B (3.6 %)	39 949 (1.5 %)		
java.lang.Class	2 758 992 B (2.3 %)	23 023 (0.9 %)		
java.util.LinkedHashMap\$Entry	1 986 520 B (1.6 %)	49 663 (1.9 %)		
char[]	1 823 544 B (1.5 %)	5 598 (0.2 %)		
java.util.HashMap	1 686 576 B (1.4 %)	35 137 (1.3 %)		
java.util.TreeMap\$KeyIterator	1 594 912 B (1.3 %)	49 841 (1.9 %)		
java.util.jar.JarFile\$JarFileEntry	1 239 784 B (1 %)	11 921 (0.5 %)		
java.util.TreeMap	1 200 576 B (1 %)	25 012 (1 %)		
java.util.concurrent.ConcurrentHashMap\$Node	1 108 160 B (0.9 %)	34 630 (1.3 %)		
java.util.ArrayList	945 528 B (0.8 %)	39 397 (1.5 %)		

Рисунок 6 – Гистограмма потребления памяти классами приложения

Чтобы понять, откуда берутся эти массивы байтов, сделал снимок памяти и отфильтровал в нём только свои классы. Там нашлись объекты сервисов приложения. Основной объём памяти уходит на массивы байтов и строки, а на собственные объекты приложения приходится совсем немного.

org.jboss.modules.Main (pid 12712)

Heap Dump

Objects ▾ | Preset: All Objects ▾ | Aggregation: [Icons] | Details: [Icons] | Preview [Icons] | Fields [Icons] | References [Icons] | GC Root [Icons] | Hierarchy [Icons]

Name	Count	Size	Retained (sort to get)
lab4.backend.api.resources.ResultResource\$Proxy\$\$_WeldSubclass	28 (0 %)	1 120 B (0 %)	n/a
lab4.backend.services.ResultService\$\$\$view2	28 (0 %)	1 120 B (0 %)	n/a
lab4.backend.services.AuthService\$\$\$view7	29 (0 %)	928 B (0 %)	n/a
lab4.backend.data.entities.ResultEntity	17 (0 %)	816 B (0 %)	n/a
lab4.backend.utils.observer.Observer\$\$\$view9	30 (0 %)	720 B (0 %)	n/a
lab4.backend.services.VersionService\$\$\$view5	28 (0 %)	672 B (0 %)	n/a
lab4.backend.api.resources.ResultResource\$1	28 (0 %)	448 B (0 %)	n/a
lab4.backend.api.utils.annotations.ApiExceptionHandler	28 (0 %)	448 B (0 %)	n/a
lab4.backend.utils.HitsCounter\$Proxy\$\$_WeldClientProxy	1 (0 %)	64 B (0 %)	n/a
lab4.backend.services.utils.annotations.ServiceExceptionHandler	4 (0 %)	64 B (0 %)	n/a
lab4.backend.services.ResultService	2 (0 %)	64 B (0 %)	n/a
lab4.backend.configuration.JWTConfig\$Proxy\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
lab4.backend.api.resources.ResultResource\$Proxy\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
lab4.backend.utils.ClickInterval\$Proxy\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
lab4.backend.data.repositories.result.postgres.ResultRepository\$\$\$view8	2 (0 %)	48 B (0 %)	n/a
lab4.backend.services.UserService\$\$\$view1	2 (0 %)	48 B (0 %)	n/a
lab4.backend.utils.HitsCounter	1 (0 %)	48 B (0 %)	n/a

Рисунок 7 – Снимок heap dump с объектами классов приложения

Поиск и устранение утечки памяти в приложении

Запустил программу и подключился к ней через VisualVM, как и в прошлый раз с бэкендом. Подождал несколько минут. Сначала посмотрел на вкладку с количеством загруженных классов и сразу заметил, что график здесь ведёт себя по-другому: число классов не выходит на плато после старта, а продолжает расти линейно всё время работы программы, без остановки.

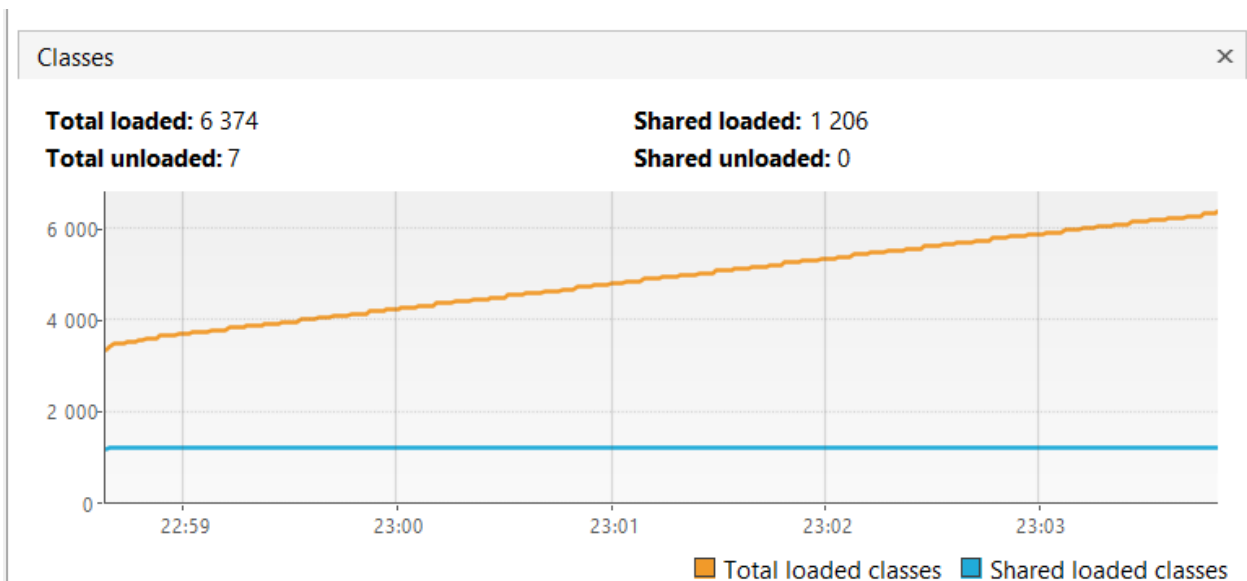


Рисунок 8 – График числа загруженных классов JVM

Дальше переключился на график потребления памяти и понаблюдал за ним подольше. Здесь видно, что размер кучи время от времени увеличивается скачками, и после очередного скачка обратно не уменьшается. При этом занятая память внутри каждого такого уровня колеблется как обычно, но сами уровни постепенно растут. Это уже похоже на утечку — программе с каждым разом требуется чуть больше памяти, чем раньше.

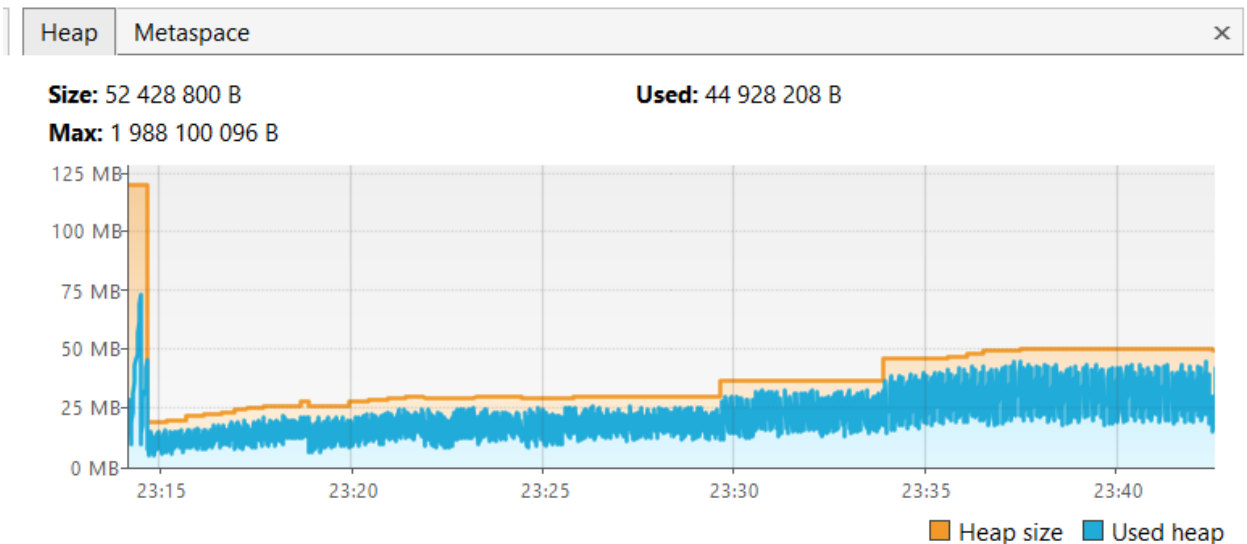


Рисунок 9 – График потребления памяти приложением

Чтобы понять, из-за чего конкретно растёт память, сделал снимок кучи, подождал несколько минут работы программы, сделал второй снимок и сравнил их между собой через сравнение снимков в VisualVM.

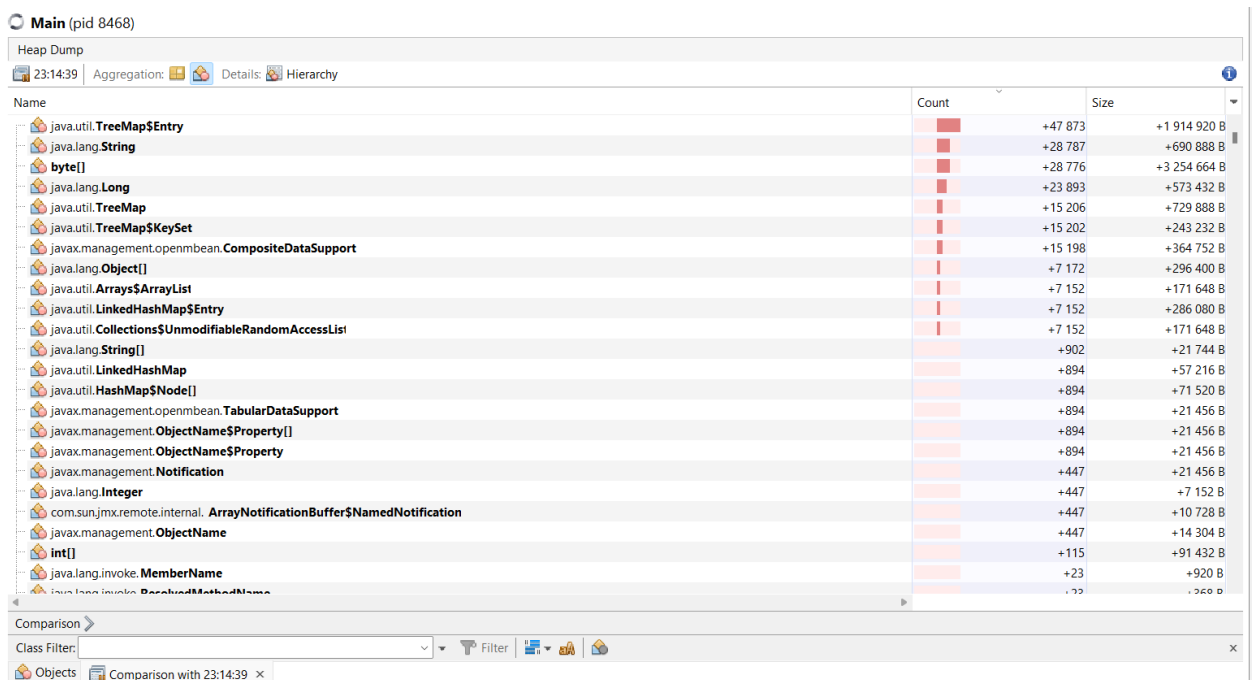


Рисунок 10 – Сравнение двух снимков памяти

В результате сравнения видно, как выросли строки и массивы байтов, причём количество новых строк за это время оказалось довольно большим, и рост не прекращается с каждым новым снимком. TreeMap находится на первом месте, но я его не учитываю, так как при анализе было выявлено что он относится к самому профилированию. Открыл список объектов типа String и стал их просматривать. Оказалось, что все новые строки представляют собой одно и то же сообщение об ошибке. То есть программа на каждой итерации создаёт ещё одну такую же строку с текстом этой ошибки, и ни одна из старых никуда не удаляется.

<instances 43 001-44 000>			
<instances 44 001-45 000>			
<instances 45 001-46 000>			
<instances 46 001-47 000>			
<instances 47 001-48 000>			
<instances 48 001-49 000>			
<instances 49 001-50 000>			
<instances 50 001-50 918>			
java.lang.String#50001: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50002: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50003: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50004: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50005: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50006: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50007: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50008: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50009: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50010: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50011: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50012: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50013: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a
java.lang.String#50014: Script 'document.wr('Hello Document')// failed: TypeError: undefined is not a function. (httpunit;)	24 B	(0 %)	n/a

Рисунок 11 – Накопленные строки с одинаковым текстом ошибки

Дальше для одной из таких строк выбрал GC Root, чтобы посмотреть, что именно держит на неё ссылку и не даёт собрать сборщику мусора. Цепочка привела к статическому полю `errorMessages` класса `JavaScript` из пакета `com.meterware.httpunit.javascript` — это список, в который складываются все сообщения об ошибках скриптов.

GC Root
Name
[4833] in java.lang.Object[]#11965: 31 618 items
elementData in java.util.ArrayList#105: 26 978 elements
static_errorMessages in class com.meterware.httpunit.javascript. JavaScript : JavaScript
[54] in java.lang.Object[]#2046: 549 items
elementData in java.util.ArrayList#104: 448 elements
classes in jdk.internal.loader.ClassLoaders\$AppClassLoader#1 [GC root - Java frame, JNI globa

Рисунок 12 – Цепочка ссылок до накопленных строк

java.lang.Object[]
[] java.lang.Object[]#11965: 31 618 items
java.lang.Object[]#2439: 2 126 items
java.lang.Object[]#3161: 1 267 items
java.lang.Object[]#360: 1 179 items
java.lang.Object[]#3181: 1 056 items
All Objects > Object[] > Object[]#11965
Class Filter:
GC Root
Name
elementData in java.util.ArrayList#105: 26 978 elements
static_errorMessages in class com.meterware.httpunit.javascript. JavaScript : JavaScript
[54] in java.lang.Object[]#2046: 549 items
elementData in java.util.ArrayList#104: 448 elements
classes in jdk.internal.loader.ClassLoaders\$AppClassLoader#1 [GC root - Java frame, JNI globa

Рисунок 13 – Поле `errorMessages` класса `JavaScript` как держатель ссылок

Поскольку это поле статическое, оно существует одно на всё время работы программы, и список внутри него никогда не очищается сам по себе. При каждом запросе в него дописывается новая строка с тем же текстом ошибки, а старые остаются лежать там же —

отсюда и бесконечный рост количества строк. Я увидел что в настройках HttpUnit есть метод для очистки сообщений об ошибках. В исходном коде программы этот метод никогда не вызывался. Добавил его вызов в основной цикл, сразу после получения ответа от сервера — теперь после каждого запроса накопленные сообщения об ошибках сбрасываются и не остаются в памяти.

```
HttpUnitOptions.setExceptionsThrownOnScriptError(true);
ServletRunner sr = new ServletRunner();
sr.registerServlet( resourceName: "myServlet", HelloWorld.class.getName());
ServletUnitClient sc = sr.newClient();
int number = 1;
WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.com/myServlet");
while (true) {
    WebResponse response = sc.getResponse(request);
    System.out.println("Count: " + number++ + response);
    java.lang.Thread.sleep( millis: 10);
    HttpUnitOptions.clearScriptErrorMessages(); // добавил вызов метода
}
```

Рисунок 14 – Исправленный код Main.java

После этого запустил программу заново и снова подключился к ней через VisualVM, чтобы проверить, помогло ли исправление. Сборщик мусора регулярно срабатывает и нормально освобождает память — никаких признаков того, что памяти со временем требуется всё больше, не видно, лишь небольшой рост от профилирования и погрешности.

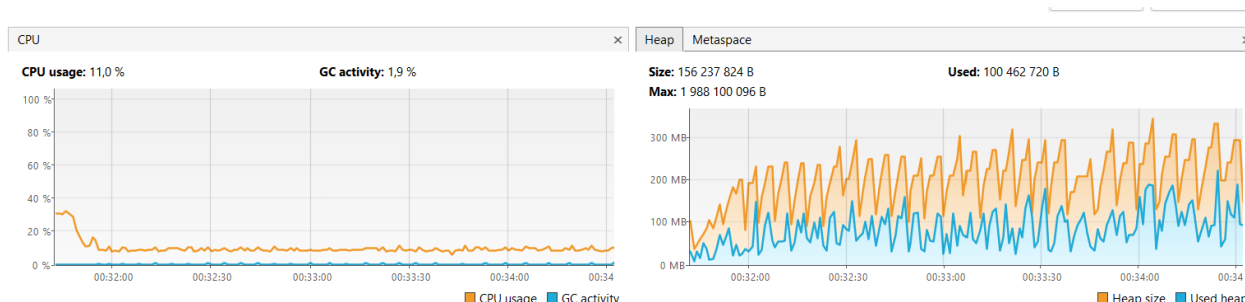


Рисунок 15 – Загрузка процессора и сборщика мусора после исправления

Чтобы убедиться окончательно, снова сделал два снимка кучи с интервалом и сравнил их так же, как делал это до исправления. На этот раз количество строк практически не изменилось между снимками — в отличие от прежнего бесконечного роста. Значит, утечка действительно была именно в этом списке сообщений об ошибках, и добавленный вызов очистки её устранил.

Main (pid 10060)		
Heap Dump		
00:15:16 Aggregation: Details: Hierarchy		
Name	Count	Size
java.lang.String	+2 802	+67 248 B

Рисунок 16 – Сравнение снимков памяти после исправления

ВЫВОД

В ходе выполнения лабораторной работы реализовал два MBean-компонента — один считает попадания и общее число точек и уведомляет о двух промахах подряд, второй вычисляет средний интервал между кликами. Через JConsole подключился к работающему серверу, посмотрел значения атрибутов этих бинов, получил уведомления о промахах и оценил число загруженных классов JVM. Через VisualVM построил графики изменения атрибутов во времени и профилировал память приложения — выяснил, что больше всего памяти занимают массивы байтов и строки, а собственные объекты приложения занимают совсем немного места. Также с помощью VisualVM нашёл и устранил утечку памяти в приложении: статический список сообщений об ошибках скриптов в библиотеке HttpUnit никогда не очищался и на каждой итерации накапливал одинаковые строки, из-за чего занятая память постепенно росла. Добавив вызов метода очистки этого списка после каждого запроса, устранил утечку, что подтвердил повторным сравнением снимков кучи — после исправления количество накопленных строк практически не менялось.